pyflam — Scientific, Technical and Operational Report

Contents

pyflam — A Scientific, Technical and Operational Report

A new, robust, multiplatform open-source tool for wildfire management, planning and suppression — coupling wildland-fire behavior science, from the Rothermel surface model through fire–atmosphere pyroconvection, into one operational pipeline.

Status: roadmap steps 1–5 implemented; step 6 (validation) ongoing. ~8,600 lines of source across 25 modules; ~568 automated tests. Pure-Python core (NumPy + SciPy); optional geospatial, atmospheric, and JIT extras; OpenFOAM and Herbie discovered at runtime. MIT-licensed; CI on Python 3.11/3.12/3.13.

Rendered versions: PDF * DOCX * Italian: PDF * DOCX * Markdown (IT).

---

0. Executive summary

pyflam is a new, robust, multiplatform open-source tool built to aid wildfire management, planning and suppression — strategic fuel and risk assessment, incident planning, and operational decision support. It is built directly from the

published, peer-reviewed wildland-fire science (Rothermel and its descendants), and is inspired by the operational paradigm that FlamMap — the USDA Forest Service / Missoula Fire Sciences Laboratory desktop system — established and proved valuable to fire agencies worldwide: a per-cell landscape product set (surface rate of spread, fireline intensity, flame length; crown-fire potential; minimum-travel-time fire growth; random-ignition burn probability). pyflam delivers that operational value as open, cross-platform, scriptable software (any OS, Python, MIT-licensed), interoperating with the same community data formats (`.lcp` landscapes, `.fms` moisture, GeoTIFF, GeoJSON) so it fits existing workflows.

On that foundation pyflam goes well beyond what the established desktop tools offer: (1) a weather-driven pipeline that derives fuel moisture and wind from live forecast/reanalysis data (GFS/ERA5/WRF) and conditions them per cell for terrain and canopy; (2) a coupled fire–atmosphere capability — native terrain-wind solvers, a buoyant-CFD plume that the fire reshapes, and ember spotting that emerges from the energy budget; (3) a literature-current crown-fire model (Cruz et al. 2005/2004) that corrects the well-documented under-prediction bias of the classic Rothermel + Van Wagner operational stack; (4) an anisotropic-Eikonal alternative to the lattice-Dijkstra spread solver that removes most of its grid bias; and (5) vertical-profile pyroconvection diagnostics (LCL, Continuous Haines, inverted-V, a Briggs plume and a pyroCb firepower threshold) that drive a spatially varying plume feedback. The value is twofold: pyflam is a *transparent, reproducible, deployable* operational tool, and a *research platform* for the fire–atmosphere coupling that the established tools approximate or omit.

The body that follows describes each capability and its scientific grounding; Appendix A gives the full physical and mathematical formulation — the energy and flux balances, the fluid-dynamics of the fire-induced plume and terrain wind, the geometric (eikonal/Finsler) propagation, the atmospheric convection and vertical-flux physics, and the moisture derivations, including how WRF/GFS/ERA5 data drive the model per cell and per timestep.

---

1. Mission and positioning

pyflam exists to put rigorous, current fire-behavior science into the hands of fire managers and planners as free, open, cross-platform software. The established desktop systems (FlamMap chief among them) demonstrated the operational value of landscape fire-behavior modelling but are closed, Windows-only binaries; pyflam was developed independently from the open, peer-reviewed fire science to deliver that value — and more — as a scriptable, automatable, deployable library that runs on any operating system, in the cloud, or inside a larger decision-support system.

Operational aims. Strategic *planning* (fuel-treatment and risk assessment via burn probability and crown-fire potential), incident *management* (near-real-time, weather-driven spread and perimeter forecasting), and *suppression* support (where the fire will go, how intense, where it will spot, and when it may go plume-dominated / pyroconvective). The same community data formats are read and written, so pyflam slots into existing GIS and fire-planning workflows rather than replacing them.

Scientific transparency and cross-validation. Every model is built from a cited publication and is unit-tested against the published equations. As an external sanity check, pyflam's deterministic outputs are cross-validated by diffing against real FlamMap rasters on a shared landscape — a convenient, widely-trusted reference, used as a benchmark, not as a specification to reproduce. (Where the established stack is known to be biased — e.g. crown-fire spread — pyflam deliberately departs from it and validates against the primary literature instead; see §6.)

The capability set pyflam provides for operational use:

Operational product	Established tools	pyflam
Surface rate of spread / intensity / flame length	Yes	`rothermel`, `landscape.basic_fire_behavior`
Crown fire potential (surface/passive/active)	Yes	`crownfire`
Minimum Travel Time fire growth	Yes	`mtt.minimum_travel_time`
Random-ignition burn probability	Yes	`mtt.burn_probability`
Landscape `.lcp` / moisture `.fms` I/O	Yes	`io_lcp`, `landscape`
Dead-fuel-moisture conditioning	Yes	`fuel_conditioning`

Everything beyond that table — Sections 5–9 — is new capability the established desktop tools do not provide.

---

2. Scientific core: the Rothermel surface model

The scientific heart of "Basic Fire Behavior" is the Rothermel (1972) surface fire-spread model with Albini's (1976) refinements: a quasi-steady energy balance in which the rate of spread is the ratio of the heat flux received by unburned fuel to the heat required for ignition,

$$R = I_R * \xi * (1 + \varphi_w + \varphi_s) / (\rho_b * \varepsilon * Q_{ig}),$$

with reaction intensity I_R, propagating flux ratio ξ, wind and slope factors φ_w, φ_s, bulk density ρ_b, effective heating number ε, and heat of pre-ignition Q_ig (Rothermel 1972). pyflam implements the full multi-size-class, multi-category (dead/live) formulation, exposing it both as a one-shot `spread(...)` call and as a reusable `SurfaceKernel` — the wind/slope-independent terms computed once per fuel + moisture and applied to scalar or array (per-cell) wind and slope. Fireline intensity follows Byram (1959): I = H * w * R; flame length follows L = 0.45*I^0.46.

Fuel inputs use the two operational standard sets: Anderson's (1982) 13 fuel models and Scott & Burgan's (2005) 40, including the dynamic live-herbaceous curing transfer. A fuel-load factor (scalar, per-fuel, or per-cell raster) scales the standard loads, which under-represent real loading by ~20–30%; because the Rothermel response is non-monotonic in load (compact litter can slow past the optimum packing ratio), this flows through the full kernel rather than scaling the output.

Validation. Against a real FlamMap run on the Tuscany landscape (1.6M cells), surface ROS matches to ~3% (regression slope 0.98, r 0.9998).

---

3. Landscape and the directional spread field

A `Landscape` is the in-memory band stack (fuel model, slope, aspect, elevation, canopy cover/height/base-height/bulk-density). pyflam reads and writes FlamMap/FARSITE `.lcp` files in pure Python (`io_lcp`, no GDAL needed) and LANDFIRE/arbitrary GeoTIFFs via rasterio, preserving the coordinate reference system for geolocation.

The transition from a *scalar* `1 + φ_w + φ_s` to a *directional* fire is roadmap step 3 and the basis of fire growth. pyflam combines the Rothermel wind factor (blowing downwind) and slope factor (pushing upslope, from the landscape aspect) as vectors, giving each cell a maximum spread rate, a heading azimuth, and an ellipse eccentricity from the Anderson (1983) length-to-breadth ratio (capped, as in FARSITE). The directional rate of spread off the heading ψ is

$$R(\psi) = R_{max} * (1 - e) / (1 - e*\cos \psi),$$

the elliptical wavelet form (Finney 1998). This `SpreadField` is the object every downstream solver consumes. *Validation:* the per-cell maximum-spread direction matches FlamMap's `MAX_SPRE_DIR` to ~1° (mean 0.96°) over 1.6M cells.

---

4. Fire growth: Minimum Travel Time, and a novel Eikonal alternative

4.1 Minimum Travel Time (the established growth engine)

Fire arrival time is the minimum-time path from the ignition over a lattice of travel directions, where the time to cross a segment is its length divided by the harmonic mean of the elliptical spread rate at its endpoints (Finney 2002). pyflam assembles the travel-time graph with vectorized NumPy (chunked, direct-CSR for huge grids) and solves the shortest path with SciPy's C-level multi-source Dijkstra, so it scales to multi-million-cell landscapes (the 5.35M-cell Tuscany landscape solves from one ignition in ~2.6 s). `max_time` bounds the search for fixed-duration runs.

4.2 The scientific limitation, and the novel fix

MTT is *Dijkstra on a lattice*, and a graph offers only as many travel directions as its neighbor template — so arrival times carry an angular (lattice) discretization bias: a calm fire becomes a faceted polygon and off-lattice bearings are biased. This is the "different calculation-point juxtapositions" Finney (2002) noted, and exactly what the numerical-analysis literature on the anisotropic Eikonal equation was built to remove. Fire arrival time satisfies a static Hamilton–Jacobi equation; the elliptical-with-wind spread law is precisely a Randers–Finsler metric (an ellipse plus a drift), as formalized for fire by Gahtan et al. (2026) and the Finsler-geodesic-spray literature.

pyflam therefore offers a selectable propagation engine: `method="mtt"` (default) or `method="fast_marching"`, a semi-Lagrangian anisotropic-Eikonal front solver that consumes the same `SpreadField`. Benchmarked against the analytic `distance / R(bearing)` on a uniform field, the Eikonal backend roughly halves MTT's bias on a wind-driven ellipse (mean ~4% vs ~12%) and beats even a denser MTT template. Three interchangeable backends (Numba-JIT, with a NumPy fallback) give identical fields; the default heap backend is a narrow-band Fast-Marching pass that prunes with **`max_time`** like Dijkstra followed by a bounding-box Gauss–Seidel correction — for a bounded single fire it is ~2x faster than MTT *and* more accurate (MTT's lattice paths are longer, so it under-predicts extent). This is, to our knowledge, a contribution not present in the operational tools: a Finsler-Eikonal fire-spread solver offered alongside, and validated against, the classical MTT.

References: Finney (2002); Sethian & Vladimirsky (2003, Ordered Upwind Methods); Mirebeau (2014, Finsler fast-marching); Gahtan, Shpund & Bronstein (2026, differentiable Randers–Finsler Eikonal solvers).

4.3 Burn probability and connected metrics

`burn_probability` reproduces a FlamMap MTT random-ignition run's full output set: burn probability plus the *connected* metrics — conditional flame length, conditional fireline intensity, the per-class flame-length probabilities (FlamMap's `FLP_METRIC`), and the per-fire size distribution. Beyond FlamMap it accepts a weather ensemble (per-fire weather variation, which is what makes the flame-length distribution non-degenerate) and solves fires in batched multi-source Dijkstra calls. The conditional fireline intensity matches FlamMap's `FIRE_LINE_INT` mean to ~2% — the strongest connected-metric agreement obtained so far.

---

5. Weather-driven fuel moisture and wind (beyond FlamMap)

FlamMap conditions dead fuel moisture from a few user-typed values; pyflam derives the moisture from weather and conditions it per cell.

5.1 Atmospheric forcing

`atmosphere` ingests forecast/reanalysis data — live GFS via Herbie (open NOAA data, no auth), ERA5 via the Copernicus CDS, or any xarray NetCDF/GRIB column — behind a provider interface, with a `ConstantAtmosphere` for testing. It carries the fire-relevant surface and convective state (wind, T, RH, surface heat flux, CAPE, CIN, PBL height) and derives pyflam's inputs: NFDRS equilibrium dead fuel moisture (Simard 1968), midflame wind, Monin–Obukhov stability, and the ambient buoyant heat flux. A time-lag (Nelson-type) `DeadFuelMoistureModel` lets the 1/10/100-h classes *remember* recent humidity rather than snapping to the latest value.

5.2 Per-cell dead-fuel-moisture conditioning

`fuel_conditioning` is the analog of FlamMap's "dead fuel moisture conditioning," grounded in a dedicated literature review (Holden & Jolly 2011; Rothermel 1983; Resco de Dios 2015; Nolan 2016). Sun-exposed fuels (south-facing, open, steep toward the sun) absorb more shortwave, run warmer than the air, and equilibrate to a *lower* moisture; shaded fuels (north-facing, under canopy, at night) stay near ambient. The module computes per-cell solar geometry (`solar_position`, with an equation-of-time / clock-time correction), a terrain-insolation factor, a canopy-transmission shading term, and conditions the moisture with either the NFDRS EMC or a semi-mechanistic VPD submodel (Resco de Dios 2015 / Nolan 2016). `condition_from_weather` is the run-setup front door: it derives the initial moisture for a date/time/location from a meteo provider (live GFS/ERA5, sampled per cell on a geolocated landscape) or, when no data exist, from manually entered T/RH. On the real Tuscany landscape this produces a ~2x

fine-fuel-moisture spread across one weather (e.g. 1.4–14% under a hot dry GFS column), instead of a single landscape-wide value.

5.3 Native terrain-wind solvers

Real terrain bends the wind; a single uniform value is a poor approximation. pyflam computes a gridded `WindField` two ways, both feeding the same midflame interface and following the WindNinja science (Forthofer et al.) implemented natively: `windsolver` — a mass-consistent solver (fast, no external dependency); and `cfd` — a momentum/RANS solver via OpenFOAM (atmospheric boundary-layer inlet, stability, diurnal slope flows, per-cell roughness from the fuel grid).

---

6. Crown fire: correcting the operational under-prediction

6.1 The problem with the FlamMap stack

FlamMap classifies crown fire with Van Wagner (1977) initiation + Rothermel (1991) active spread (R_active = $3.34R_{10}$) + *Scott & Reinhardt (2001) surface/passive/active classification. Cruz & Alexander (2010) showed that this operational stack — naming FlamMap, FARSITE, NEXUS, BehavePlus and FFE-FVS explicitly — carries a significant crown-fire-spread under-prediction bias, from three sources: incompatible surface -> crown model linkages, the inherent under-prediction of the Rothermel ROS models, and an* unsubstantiated* reduction of crown spread by crown fraction burned. The Van Wagner *initiation* physics is sound; the *spread* side is outdated.

6.2 The pyflam approach

pyflam keeps the FlamMap stack as the default but makes the active-spread model selectable (`crown_spread="rothermel1991" | "cruz2005"`). The Cruz, Alexander & Wakimoto (2005) model — `CROS_active = 11.02*U10^0.90*CBD^0.19*exp(-0.17*M)` (verified against the primary paper) — is the validated successor; on the Cruz path an active crown fire spreads at the *full* Cruz rate, dropping the unsubstantiated crown-fraction reduction. A CFIS logistic crown-fire-initiation model (Cruz et al. 2004, coefficients verified at source) provides a *probability* of crowning as an alternative to the deterministic Van Wagner threshold. On a real GEDI-derived canopy landscape (Section 10) the Cruz model classifies markedly more cells as active crown fire than Rothermel — the under-prediction made visible.

Because FlamMap and the whole Rothermel+Van Wagner stack are *known to under-predict* crown fire, pyflam deliberately does not validate crown fire against FlamMap (that would validate toward a biased reference). Instead its crown fire is *faithful to the literature-validated Cruz models* (unit-tested against their published coefficients), so it inherits their validation against observed wildfires (Cruz 2005: ~61% of variance over 57 wildfire observations).

---

7. Ember spotting from the energy budget

`spotting` provides two firebrand models that both couple into MTT growth and burn probability. `SpottingModel` is a fast parameterized loft-and-drift model. The novel one is `FirebrandPhysics`: a stochastic, physics-based model in which spotting *emerges* from the energy system — buoyant-plume loft from fireline intensity (Morton–Taylor–Turner), firebrand size -> terminal velocity (drag) -> loft and combustion burnout (Tarifa's d^2-law), downwind transport, and a landing-ignition probability that falls with the receiving fuel's moisture. Randomness (Poisson brand counts proportional to intensity, lognormal sizes, turbulent bearing, Bernoulli ignition) makes the landing pattern a Monte-Carlo outcome; the constants are *physical*, tied to measured firebrand data (Tohidi & Kaye and Manzello terminal velocities; Manzello size distribution), not reach calibrations. The one weakly-constrained length is calibrated against literature spot-distance anchors.

---

8. Coupled fire–atmosphere: pyroconvection and crown–plume–spotting feedback

This is pyflam's most distinctive scientific contribution and is absent from the operational tools.

8.1 The plume the fire makes

`pyroconvection` turns the fire's fireline intensity into a convective ground heat-flux field, runs a buoyant RANS (OpenFOAM `buoyantBoussinesqSimpleFoam`), and returns the wind *including the fire's own plume* (indrafts/updraft) — the feedback in which the fire reshapes the wind that drives it (verified end-to-end: a hot patch shifts the mean near-surface wind 2.43 -> 3.14 m/s). `fire_atmosphere_march` time-marches this coupling, re-solving the plume wind every `dt` minutes of MTT growth, with the wind solver injectable so the loop is usable and testable without OpenFOAM.

8.2 Crown -> plume -> wind -> crown feedback

The regime where the plume matters most is crown fire. pyflam closes a genuinely novel loop (`docs/crown_plume_coupling.md`): with `crown=True` each march step rebuilds a crown-aware spread field from the current plume-modified wind — crowning cells spread at the Cruz rate and carry the much-higher crown intensity, which feeds straight into the plume solver *and* the ember spotting, closing crowning -> stronger plume -> higher wind -> faster crown spread. The positive feedback is bounded by under-relaxation of the wind and a physical cap (a synthetic stress test confirms no runaway). On real data the crown intensity ran ~40x the surface value, giving far greater spotting reach.

8.3 Vertical-profile pyroconvection diagnostics

A dedicated review of vertical/drift wind and the boundary-layer/condensation-level relationship established the key, counter-intuitive result: deep convective (pyroCb) fire is a vertical problem — a deep, dry, well-mixed boundary layer (high LCL) capped by moisture aloft (the "inverted-V" sounding) with elevated lower-tropospheric instability — not high surface CAPE (pyroCb routinely form with near-zero surface CAPE; mid-level humidity is the discriminator). pyflam implements diagnostics keyed on that geometry: `lcl_height_m`, the Continuous Haines index (Mills & McCaw 2010), an inverted-V detector, and `pyroconvection_potential` (Castellnou et al. 2022; Peterson et al. 2017). It adds a Briggs (1969) bent-over plume — the validated wildfire-plume form (Lareau & Clements 2017) — and a PyroCb Firepower Threshold (Tory & Kepert 2021): the minimum firepower for the plume to reach condensation against the capping stability.

These are wired into the coupled march: with `pyroconvection=True` the plume intensity is scaled by a profile-aware `convective_plume_factor`, so a dry/unstable inverted-V atmosphere drives a stronger plume and a stable one damps it — and in `spatial=True` mode the factor is per cell, so under one moist-aloft column the locally dry (high-LCL) cells get the pyroconvective boost while moist cells do not. This re-weights the convective coupling toward the predictors the pyroCb literature favors over surface CAPE.

---

9. Operational products and near-real-time use

pyflam is engineered for analyst-facing operation, not only research:

- **`meteo_report`** — a near-real-time fire-weather variation report sampling the atmosphere across the run window, tracking how T, RH, wind, dead fuel moisture (per time lag), convective state (CAPE/CIN/PBL/stability) and energy fluxes *change*.
- **`operative`** — operational analysis of a run perimeter: it splits the perimeter into head / flanks / tail (or finer sub-sectors) and decomposes the spread drive into slope, fuel (gradient of intrinsic fireline intensity) and wind vectors plus their resultant and dominant driver — the "arrows" a map front-end draws — with GeoJSON export (sector forces, contour-traced perimeter polygon, reprojected to WGS84).
- **`nrt.run_realtime`** — a one-call near-real-time product tying weather -> time-marched spread -> perimeter -> both reports into a single `RunProduct`.

An end-to-end operational scenario is reproducible via `tests/final_run_tuscany.py`: 300 random ignitions on the real 5.35M-cell Tuscany landscape, 28 June 2026, a 36-hour fire driven by live GFS weather with per-cell conditioned moisture, writing each pipeline phase to its own structured output folder (landscape -> weather -> moisture -> surface behavior -> wind -> spread field -> ignitions -> burn probability + metrics).

---

10. Real-data canopy fuels (GEDI) and the crown pipeline

No global canopy-base-height / bulk-density product exists — GEDI and the GEDI-calibrated canopy-height maps give *height* only; CBH/CBD are either US-LANDFIRE or must be derived. pyflam includes a reproducible path (`tests/fetch_canopy_tuscany.sh`, `tests/build_canopy_landscape_tuscany.py`) that downloads GEDI-calibrated canopy height (Meta/WRI High-Resolution Canopy Height, Tolan et al. 2024, open AWS bucket), warps it onto the Tuscany grid, and derives CBH/CBD from height + canopy cover with transparent, documented fire-science heuristics (crown ratio rising with cover -> lower crown base; CBD = cover-scaled load / canopy depth). The result is a full canopy `.lcp` on which the entire crown pipeline runs — read -> Cruz vs Rothermel classification -> crown-aware spread field -> plume-coupled crown march. (CBH/CBD here are *derived estimates*, clearly flagged as such, not field measurements.)

---

11. Validation strategy

The acceptance test is a cell-by-cell diff against real FlamMap output, not internal self-consistency. `validate` provides the generic machinery — robust field comparison (bias/RMSE/ratios/correlation/OLS, log-space and "within-X%" stats, burn/no-burn classification), perimeter overlap (Jaccard/Dice/Hausdorff), arrival-time agreement, and categorical agreement (confusion matrix + per-class recall, for crown-fire type) — with data-wiring scripts per landscape (`tests/validate_flammap_*.py`).

Validated to date (Tuscany, 1.6M cells): surface ROS ~3%, max-spread direction ~1°, conditional fireline intensity ~2% vs FlamMap. Partially recoverable: burn probability (Monte-Carlo and spotting-parameter limited). Open: a crown-fire diff against *observed* (not FlamMap) ROS, and a spotting-off single-fire perimeter / time-of-arrival diff (the bundled dataset ships no arrival-time raster). The two test layers — physics/property tests asserting known relationships, and golden-master regressions locking current numerics — total ~568 automated tests run in CI across Python 3.11/3.12/3.13.

---

12. Software engineering

Pure-Python core on NumPy + SciPy; optional extras gate heavier capabilities — `geo` (rasterio/pyproj/scikit-image), `atmos` (xarray/cfgrib/netcdf4/cdsapi), `accel` (Numba JIT for the Eikonal solver). External engines (OpenFOAM, Herbie/ERA5) are discovered at runtime and their tests self-skip when absent, so the core installs and runs anywhere. The vectorized kernel design (a `SurfaceKernel` computed once and applied to per-cell array inputs) is what makes whole-landscape moisture conditioning, weather forcing and crown classification tractable at multi-million-cell scale. CI is SHA-pinned to Node-24 actions; coverage is reported to Codecov.

---

13. Summary of novel contributions beyond the established tools

1. Weather-driven, per-cell fuel moisture — live GFS/ERA5 -> terrain-insolation + canopy-shading conditioning (EMC or VPD), vs FlamMap's typed scalars.
2. Selectable anisotropic-Eikonal spread engine — a Finsler-Eikonal solver alongside MTT, removing most of the lattice bias and, in heap form, beating Dijkstra on bounded fires in both speed and accuracy.
3. Burn probability with a weather ensemble and the full connected-metric set.
4. Literature-current crown fire — Cruz 2005 active spread + Cruz 2004 logistic initiation, correcting the documented under-prediction of the operational stack.
5. Coupled fire–atmosphere — native terrain winds, a buoyant-CFD plume the fire reshapes, physics-based spotting, and a crown -> plume -> wind -> crown feedback.
6. Vertical-profile pyroconvection — LCL/C-Haines/inverted-V diagnostics, a Briggs plume and a pyroCb fire-power threshold, driving a per-cell plume feedback keyed on the ABL/LCL geometry rather than surface CAPE.
7. Operational NRT layer — weather-variation and perimeter-driver reports with GeoJSON export, plus a one-call real-time product.

The scientific value is that each of these is grounded in the peer-reviewed literature and implemented transparently and reproducibly. Where a well-established model is the operational standard (e.g. Rothermel surface spread, the classic crown stack) it is retained as a trusted default; the novel methods are offered as validated, selectable alternatives — so pyflam is at once a dependable operational tool and a platform for advancing the science.

References

- Albini, F.A. (1976). *Estimating wildfire behavior and effects.* USDA FS GTR INT-30.
- Anderson, H.E. (1982). *Aids to determining fuel models for estimating fire behavior.* USDA FS GTR INT-122.
- Anderson, H.E. (1983). *Predicting wind-driven wildland fire size and shape.* USDA FS RP INT-305.
- Briggs, G.A. (1969). *Plume Rise.* USAEC TID-25075.
- Byram, G.M. (1959). *Combustion of forest fuels.* In *Forest Fire: Control and Use.*
- Castellnou, M.; Stoof, C.R.; Vilà-Guerau de Arellano, J.; et al. (2022). *Pyroconvection classification based on atmospheric vertical profiling.* J. Geophys. Res. Atmos. 127, e2022JD036920.
- Cruz, M.G.; Alexander, M.E.; Wakimoto, R.H. (2004). *Modeling the likelihood of crown fire occurrence in conifer forest stands.* Forest Science 50(5), 640–658.
- Cruz, M.G.; Alexander, M.E.; Wakimoto, R.H. (2005). *Development and testing of models for predicting crown fire rate of spread.* Can. J. For. Res. 35(7), 1626–1639.
- Cruz, M.G.; Alexander, M.E. (2010). *Assessing crown fire potential in coniferous forests of western North America: a critique of current approaches.* Int. J. Wildland Fire 19, 377–398.
- Finney, M.A. (1998). *FARSITE: Fire Area Simulator — model development and evaluation.* USDA FS RP RMRS-RP-4.
- Finney, M.A. (2002). *Fire growth using minimum travel time methods.* Can. J. For. Res. 32(8), 1420–1424.
- Forthofer, J.M.; et al. (WindNinja). *Mass-consistent and momentum diagnostic wind models for wildland fire.*
- Gahtan, B.; Shpund, J.; Bronstein, A.M. (2026). *Wildfire Simulation with Differentiable Randers–Finsler Eikonal Solvers.* arXiv:2603.00035.
- Holden, Z.A.; Jolly, W.M. (2011). *Modeling topographic influences on fuel moisture and fire danger in complex terrain.* Forest Ecology and Management 262, 2033–2041.
- Lareau, N.P.; Clements, C.B. (2017). *The Mean and Turbulent Properties of a Wildfire Convective Plume.* J. Appl. Meteorol. Climatol. 56(8).
- Manzello, S.L.; et al. *Firebrand (ember) size and generation measurements.*
- Mills, G.A.; McCaw, W.L. (2010). *Atmospheric stability environments and fire weather in Australia — the Continuous Haines index.* CAWCR Tech. Rep. 20.
- Mirebeau, J.-M. (2014). *Efficient fast marching with Finsler metrics.* Numerische Mathematik.
- Morton, B.R.; Taylor, G.; Turner, J.S. (1956). *Turbulent gravitational convection from maintained and instantaneous sources.* Proc. R. Soc. Lond. A.
- Morvan, D.; Frangieh, N. (2018). *Wildland fires behaviour: wind effect versus Byram's convective number.* Int. J. Wildland Fire 27(10).
- Nelson, R.M. (2000). *Prediction of diurnal change in 10-h fuel stick moisture content.* Can. J. For. Res. 30, 1071–1087.
- Nolan, R.H.; Resco de Dios, V.; Boer, M.M.; et al. (2016). *Predicting dead fine fuel moisture at regional scales using vapour pressure deficit.* Remote Sensing of Environment 174, 100–108.
- Peterson, D.A.; et al. (2017). *Pyrocumulonimbus climatology — mid-troposphere humidity as a pyroCb discriminator.*
- Resco de Dios, V.; et al. (2015). *A semi-mechanistic model for predicting the moisture content of fine litter.* Agricultural and Forest Meteorology 203, 64–73.
- Rothermel, R.C. (1972). *A mathematical model for predicting fire spread in wildland fuels.* USDA FS RP INT-115.
- Rothermel, R.C. (1983). *How to predict the spread and intensity of forest and range fires.* USDA FS GTR INT-143.
- Rothermel, R.C. (1991). *Predicting behavior and size of crown fires in the Northern Rocky Mountains.* USDA FS RP INT-438.
- Scott, J.H.; Reinhardt, E.D. (2001). *Assessing crown fire potential by linking models of surface and crown fire behavior.* USDA FS RMRS-RP-29.
- Scott, J.H.; Burgan, R.E. (2005). *Standard fire behavior fuel models.* USDA FS GTR RMRS-GTR-153.
- Sethian, J.A.; Vladimirsky, A. (2003). *Ordered Upwind Methods for Static Hamilton–Jacobi Equations.* SIAM J.

Numer. Anal. 41(1), 325–363.
- Simard, A.J. (1968). *The moisture content of forest fuels.* Canadian Dept. of Forestry.
- Tarifa, C.S.; et al. (1965). *On the flight paths and lifetimes of burning particles of wood.* Proc. Combustion Institute.
- Tohidi, A.; Kaye, N.B. *Aerodynamic characterization of firebrands / terminal velocity.*
- Tolan, J.; et al. (2024). *Very high resolution canopy height maps from RGB imagery (Meta/WRI HRCH).* Remote Sensing of Environment.
- Tory, K.J.; Kepert, J.D. (2021). *Pyrocumulonimbus Firepower Threshold: Assessing the Atmospheric Potential for pyroCb.* Weather and Forecasting 36(2).
- Van Wagner, C.E. (1977). *Conditions for the start and spread of crown fire.* Can. J. For. Res. 7(1), 23–34.

Appendix A — Physical and mathematical formulation

The governing equations behind each layer, in the form pyflam implements them. Notation: g gravitational acceleration, ρ density, c_p specific heat, T temperature (K unless noted), θ potential temperature, p pressure, U wind speed, u friction velocity, q" a flux per unit area (W m^{-2}), I a fireline intensity per unit length (W m^{-1}), R a rate of spread.*

A.1 Energy balance and heat fluxes — the surface fire

Rothermel's spread rate is a steady-state energy balance: the rate of spread is the heat flux received by the unburned fuel divided by the heat required to bring it to ignition,

$$R = \frac{I_R * \xi * (1 + \varphi_w + \varphi_s)}{\rho_b * \varepsilon * Q_{ig}} \qquad [\mathrm{m\ s^{-1}\ or\ ft\ min^{-1}}]$$

- Reaction intensity I_R (kW m^{-2} ; the energy-release rate per unit area of the flaming front) = Γ' * w_n * h * η_M * η_s, with the optimum reaction velocity Γ', net fuel load w_n, low heat content h, and the moisture and mineral damping coefficients η_M, η_s. Moisture enters *here*, nonlinearly, through η_M(M/M_x) — which is why pyflam applies per-cell moisture inside the kernel rather than to the output.
- Propagating flux ratio ξ — the fraction of I_R that reaches the fuel ahead, a function of packing ratio β and surface-area-to-volume ratio σ.
- Wind and slope factors $\varphi_w = C(\beta U)^{B(\beta/\beta_{op})^{(-E)}}$, $\varphi_s = 5.275\beta^{(-0.3)} tan^2(slope)$ — *multiplicative enhancements of the no-wind, no-slope rate* $r_0 = I_R\xi/(\rho_b\varepsilon Q_{ig})$.
- Heat sink ρ_bεQ_ig — bulk density x effective heating number ε = exp(−138/σ) x heat of pre-ignition Q_ig = 250 + 1116*M (kJ kg^{-1}), the term moisture raises.

The fire's power is the Byram (1959) fireline intensity, an energy flux per unit length of front,

$$I = H * w * R \qquad [\mathrm{W\ m^{-1}}]$$

flame length $L = 0.0775 * I^{0.46}$ [m]

with H heat yield and w fuel consumed. Two heat-transfer pathways carry I to the unburned fuel: radiation (dominant in plume-dominated fire) and convection (dominant when wind tilts the flame forward). pyflam tracks I per cell on the `SpreadField`; it is the quantity that subsequently drives the plume (§A.3) and ember lofting (§7).

A.2 Geometric propagation — Huygens, the eikonal, and the Finsler metric

Once each cell has a directional spread rate R(ψ) = R_max(1−e)/(1−e*cosψ) (an ellipse, Finney 1998), fire growth is a wavefront-propagation problem with two equivalent formulations:

Huygens / Hamilton–Jacobi (continuous front). Every point of the perimeter is a source of an elliptical wavelet; the new front is the envelope. The arrival-time field T(x) then satisfies a static, anisotropic Hamilton–Jacobi (eikonal) equation

$$F(x, \nabla T/|\nabla T|) * |\nabla T| = 1 , \qquad T = 0 \text{ on the ignition,}$$

where F is the front normal speed. When F depends on direction (always, under wind and slope) the equation is *anisotropic*; for the wind-tilted ellipse it is *asymmetric* (downwind != upwind) — i.e. a Finsler metric whose indicatrix (unit ball) is the cell's fire ellipse. Arrival time is the geodesic distance in that metric.

Minimum Travel Time (discrete graph). Discretize space as a lattice graph whose edge weights are travel times $\Delta x/R(\psi)$; arrival time is the shortest path (Dijkstra), Finney (2002). This is exact *as a graph* but offers only the template's discrete directions, so it carries an O(angular-resolution) metrication error — the lattice bias §4.2 quantifies.

pyflam's `fast_marching` backend solves the eikonal form directly with a semi-Lagrangian update: the time at a cell is

```
T(x) = min over directions  [ τ(x, y) + interpolated T on the segment through y ],
```

minimised over a continuum of arrival directions (not just lattice nodes), with τ the Finsler travel time. A causal, heap-ordered single pass (the anisotropic-Eikonal analogue of Dijkstra; Sethian & Vladimirsky 2003, Mirebeau 2014) removes most of the metrication error and lets a finite horizon prune the front exactly as MTT does.

A.3 Fire-induced fluid dynamics — buoyant plume and mass-consistent wind

The plume (momentum + buoyancy, RANS). The fire injects a convective ground heat flux, cell-averaged from the fireline intensity over the cell,

```
q''_fire = χ_c ∗ I[W m⁻¹] / Δx           [W m⁻²]   (pyflam: χ_c  ~  0.6 convective fraction)
```

restricted to actively flaming cells. This forces a steady, buoyant Reynolds-averaged Navier–Stokes system under the Boussinesq approximation (OpenFOAM `buoyantBoussinesqSimpleFoam`):

```
grad ∗u = 0                                                          (mass)
grad ∗(u u) = −grad p_rgh/ρ₀ − g∗(ρ−ρ₀)/ρ₀ k_hat + grad ∗[(ν+ν_t)grad u]     (momentum + buoyancy)
grad ∗(u T) = grad ∗[(α+α_t)grad T] + q''_fire/(ρ₀ c_p)                (energy / heat)
```

with k–ε turbulence closure and an atmospheric-boundary-layer log-law inlet. The buoyancy term $g*(\rho-\rho_0)/\rho_0$ is the plume engine: hot, light air over the fire rises, drawing an indraft at the surface and a return flow aloft. The solver returns a plume-modified `WindField` that feeds back into spread — the coupling FlamMap omits.

Plume rise (Briggs, bent-over). For the observed bent-over wildfire plume in a crosswind the rise scales with the buoyancy flux

```
F = g∗Q_c / (π∗ρ∗c_p∗T)     [m⁴ s⁻³] ;   stable cap:  Δh = 2.6∗(F/(U∗s))^{1/3}
```

$s = (g/\theta)*d\theta/dz$ the static-stability (squared Brunt–Väisälä) parameter. Inverting Δh for Q_c gives the pyroCb fire-power threshold (§8.3): the least fire power that lifts the plume to condensation against the cap.

Mass-consistent terrain wind (diagnostic, no momentum). Where the full RANS is too costly, pyflam uses the Sasaki (1970) variational method: find the wind nearest an interpolated first guess u_0 that is divergence-free. With a Lagrange multiplier λ this reduces to an anisotropic Poisson problem,

```
d²λ/dx² + d²λ/dy² + T_R∗d²λ/dz² = −2 grad ∗u₀ ,    u = u₀ + 1/2grad _h λ ,  w = w₀ + (T_R/2)∗λ_z
```

(T_R the stability/anisotropy ratio) — ridge speed-up, valley channeling and lee deceleration emerge from mass conservation alone (Forthofer et al. 2014).

A.4 Atmospheric physics — vertical fluxes, convection, and the ABL/LCL geometry

Surface-layer fluxes (Monin–Obukhov similarity). The atmosphere's own turbulent exchange sets the background into which the plume grows. From the surface sensible heat flux q''_H and the friction velocity $u^* = \kappa U/\ln((z+z_0)/z_0)$,

```
L = −u∗³ ρ c_p T / (κ g q''_H)        (Obukhov length)
```

$L < 0$ unstable (daytime convective, plume grows freely), $L > 0$ stable (capped). pyflam classifies stability from the sign of q''_H (with a CAPE/CIN override) and damps or boosts the convective plume factor accordingly.

Moist convection and the vertical profile. Whether the plume merely rises or deepens into pyrocumulus/pyrocumulonimbus is governed by the *vertical thermodynamic structure*, not surface buoyancy. Key quantities, all computed from a column profile:

- Potential temperature θ = T*(1000/p)^0.286 — conserved in dry adiabatic ascent; a well-mixed boundary layer has θ ~ const (s ~ 0), so the plume rises unimpeded to the condensation level.
- Lifting condensation level LCL — the height a surface parcel must be lifted to saturate; pyflam uses LCL ~ 125*(T − T_d) m (Espy/Lawrence), with the dewpoint T_d from the inverse Magnus relation. A hot, dry mixed layer => large dewpoint depression => high LCL.
- CAPE / CIN — the buoyant energy released / the inhibition to overcome. Crucially, pyroCb routinely form with near-zero surface CAPE: in the canonical "inverted-V" sounding (deep dry mixed layer capped by moisture aloft) the parcels that matter originate in the hot dry layer and find moisture *above* the LCL. The discriminator is therefore mid-tropospheric humidity, not surface CAPE.
- Continuous Haines C-Haines = CA(stability, 850 -> 700 hPa lapse) + CB(dryness, 700-hPa dewpoint depression) — the operational lower-tropospheric instability+dryness index.

pyflam's `pyroconvection_potential` flags a column as plume-favourable when the boundary layer is deep and dry (high LCL) and there is moisture/instability aloft (inverted-V or high C-Haines) — the ABL -> LCL -> free-convection geometry the pyroCb literature identifies (Castellnou 2022; Peterson 2017; Tory & Kepert 2021).

A.5 Moisture derivations — equilibrium, VPD, time lag, and per-cell conditioning

Equilibrium moisture content (instantaneous). The moisture a dead fuel approaches in constant air is the NFDRS piecewise EMC m_e(T, RH) (Simard 1968), rising with RH and falling weakly with T. The semi-mechanistic alternative ties fine-fuel moisture to vapour-pressure deficit, M = a + b*exp(−c*VPD) (Resco de Dios 2015 / Nolan 2016 Eq. 8: a=7.86, b=140.94, c=3.73; VPD in kPa), with VPD = e_s(T)*(1 − RH/100).

Time-lag dynamics (temporal memory). Real fuels lag the air; each size class relaxes toward EMC by a first-order ODE

`dm/dt = (m_e − m)/τ  =>  m(t+Δt) = m_e + (m(t) − m_e)*e^{−Δt/τ}`

with τ = 1, 10, 100 h. pyflam's `DeadFuelMoistureModel` integrates this across march timesteps, so the 1/10/100-h moistures carry memory of recent humidity rather than snapping to the latest value — essential for diurnal drying and multi-hour reanalysis runs.

Per-cell terrain/canopy conditioning. The same air produces very different fuel moisture on a sunlit south slope than in a shaded draw. pyflam computes, per cell, a sun-exposure index S in [0,1] = (direct-beam factor on the slope facet) x (canopy openness):

```
cos(incidence) = cos(slope)cos(zenith) + sin(slope)sin(zenith)cos(sun_az − aspect)
S = max(cos incidence, 0) * (1 − shade * canopy_cover)
```

with the solar zenith/azimuth from latitude, day-of-year and (equation-of-time-corrected) hour. A near-fuel heating ΔT = ΔT_sun*S is added; holding the surface vapour pressure fixed, the warmer fuel sees a lower local RH = 100*e_vap/e_s(T+ΔT), and the EMC (or VPD) is evaluated at that warmer, drier microclimate. Sun-exposed cells thus dry below ambient, shaded cells stay near it — a ~2x fine-fuel-moisture spread across one landscape under identical weather (Holden & Jolly 2011; Rothermel 1983).

A.6 WRF/GFS/ERA5 forcing — per-cell sampling at temporal scale

pyflam consumes numerical-weather-prediction and reanalysis data through one provider interface, with three coupled scales of resolution:

- Per-cell (spatial). `field_on(ls, time)` samples the gridded column onto every landscape cell (nearest-neighbour in the source grid, longitudes wrapped to the source convention — GFS 0–360°, ERA5 −180–180°), so wind, temperature, humidity and the derived dead fuel moisture vary across the domain. On a geolocated landscape the cell-centre latitudes/longitudes are reprojected from the landscape CRS (e.g. ETRS89-LAEA) to geographic.

- Per-timestep (temporal). `fire_atmosphere_march` re-reads the provider at the advancing clock time every Δt minutes, so spread, plume and moisture respond to evolving weather — a forecast (GFS forecast hour fxx) or a reanalysis (ERA5 hourly) — and the time-lag moisture (§A.5) carries state between steps.
- Flux unit reconciliation. ERA5 surface fluxes are *accumulated* (J m^{-2} over the product step) and positive *downward*; pyflam converts them to instantaneous W m^{-2} positive *upward* (q'' = −J m^{-2} / Δt_accum) so the sign and units match the buoyant background of §A.3/A.4. WRF/GFS instantaneous fluxes pass through directly.

Concretely, the end-to-end Tuscany run (§9) pulls a live GFS column for 28 June 2026, derives T=36.5 °C / RH=25 % at the site, conditions the dead fuel moisture per cell to 1.4–14 % across 5.35 million cells by terrain insolation, and feeds that — with the GFS wind and, optionally, the inverted-V plume boost (§8.3) — into the MTT/Eikonal spread and burn-probability solvers. Every physical layer above is exercised in that single run, on real data, at the native landscape resolution.

AI-assistance disclosure

This report, and substantial parts of the pyflam implementation it describes, were produced with the assistance of an AI coding/research tool (Claude). The underlying fire-science models are independent implementations of the cited peer-reviewed publications; the literature reviews that motivated the novel components were conducted with AI-assisted search, and every cited source was located in live searches and (where central) verified at its primary source. Coefficients of the empirical models were checked against the original papers before implementation.